

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Aceves Siordia	05/10/2025
	Nombre: Kevin Damian	

Actividad 1: Creación de la aplicación para gestión financiera en contenedores en la plataforma Docker

Introducción

El presente informe documenta la actividad practica que tiene como objetivo crear, configurar e implementar en contenedores Docker el despliegue de una aplicación full-stack, en mi caso he optado por desplegar en 3 contenedores un CMS (Content Management Service) llamado WordPress.

Descripción y diseño de la solución

El entorno por desplegar constara de tres contenedores conectados mediante una red bridge de Docker. La separación se realizo por “capa”, es decir, cada imagen es responsable de una responsabilidad

- WordPress (PHP-FPM)

Representa la capa de la aplicación y lógica de negocio. Se construyo a partir de la imagen oficial de PHP-FPM a la cual se le agrego la descarga e instalación de WordPress y WordPress CLI. Esta configurada para recibir mediante variables de entorno o archivo de configuración los valores necesarios para conectarse y/o crear la base de datos.

- MySQL

Funciona como la base de datos. Se emplea la imagen oficial de MySQL, manteniendo su configuración por defecto salvo por pasar mediante variables de entorno la configuración de credenciales y nombre. Además, se le asocio un volumen para garantizar la durabilidad de los datos sin importar si el contenedor se destruye

- Nginx

Actúa como servidor HTTP y proxy inverso, reenviando las peticiones dinámicas al contenedor de WordPress, ya que PHP-FPM no soporta ser un servidor web. Para este se construyo una imagen personalizada con una configuración precargada del nginx.conf.

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Aceves Siordia	05/10/2025
	Nombre: Kevin Damian	

Asegurando que esta lista desde el arranque, pero aun dejando la posibilidad a ser cambiada

Implementación

Para el contenedor de WordPress, se tomó como base la imagen de php-fpm, la cual es ideal para montar un WordPress sobre él; a continuación, se muestra el Dockerfile con comentarios sobre lo que hace cada línea

```

1  # Imagen base de PHP con FPM
2  FROM php:8.2-fpm
3
4  # Instalar paquetes requeridos
5  RUN apt-get update && apt-get install -y \
6      default-mysql-client \
7      libzip-dev \
8      libonig-dev \
9      libxml2-dev \
10     curl \
11     tar \
12     unzip \
13     && docker-php-ext-install mysqli pdo pdo_mysql zip \
14     && rm -rf /var/lib/apt/lists/*
15
16 # Crear directorio para WordPress
17 RUN mkdir -p /var/www && chown www-data:www-data /var/www
18
19 # Descargar WordPress
20 RUN curl -sSL https://wordpress.org/latest.tar.gz | tar -xz -C /var/www && \
21     chown -R www-data:www-data /var/www/wordpress
22
23 # Instalar WP-CLI
24 RUN curl -O https://raw.githubusercontent.com/wp-cli/builds/gh-pages/phar/wp-cli.phar \
25     && chmod +x wp-cli.phar \
26     && mv wp-cli.phar /usr/local/bin/wp
27
28 # Copiar entrypoint y hacerlo ejecutable
29 COPY docker-entrypoint.sh /usr/local/bin/docker-entrypoint.sh
30 RUN chmod +x /usr/local/bin/docker-entrypoint.sh
31
32 # Establecer el usuario del contenedor a www-data (en lugar de root)
33 USER www-data
34
35 # Establecer directorio de trabajo
36 WORKDIR /var/www/wordpress
37
38 # Exponer el puerto 9000
39 EXPOSE 9000
40
41 # Indicar que el punto de entrada es el script
42 ENTRYPOINT ["docker-entrypoint.sh"]
43

```

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Aceves Siordia	05/10/2025
	Nombre: Kevin Damian	

Como se puede observar, el Dockerfile finaliza con la ejecución de un script de *entrypoint*. Esto aporta a la flexibilidad cuando se despliega, ya que permite realizar la configuración de WordPress de dos formas:

- Un archivo wp-config.php, que contendrá los parámetros de conexión a la base de datos y la información del sitio.
- Mediante variables de entorno, definidas durante el arranque del contenedor, lo que permitirá adaptar la configuración a distintos entornos.

```

You, 16 minutes ago | 1 author (You)
1  #!/bin/bash
2  set -e
3
4  # Asignar valores por defecto si no existen
5  # Base de datos
6  DB_NAME=${DB_NAME:-wordpress}
7  DB_USER=${DB_USER:-wpuser}
8  DB_PASSWORD=${DB_PASSWORD:-wppassword}
9  DB_HOST=${DB_HOST:-wp-db}
10
11 # Configuración de WordPress
12 WP_URL=${WP_URL:-http://localhost:8080}
13 WP_TITLE=${WP_TITLE:-"Mi WordPress"}
14 WP_ADMIN_USER=${WP_ADMIN_USER:-admin}
15 WP_ADMIN_PASSWORD=${WP_ADMIN_PASSWORD:-adminpassword}
16 WP_ADMIN_EMAIL=${WP_ADMIN_EMAIL:-kevin.aceves@hotmail.com}
17
18 # Copiar wp-config.php si no existe
19 if [ ! -f /var/www/wordpress/wp-config.php ]; then
20     cp /var/www/wordpress/wp-config-sample.php /var/www/wordpress/wp-config.php
21     sed -i "s/database_name_here/$DB_NAME/" /var/www/wordpress/wp-config.php
22     sed -i "s/username_here/$DB_USER/" /var/www/wordpress/wp-config.php
23     sed -i "s/password_here/$DB_PASSWORD/" /var/www/wordpress/wp-config.php
24     sed -i "s/localhost/$DB_HOST/" /var/www/wordpress/wp-config.php
25
26     # Instalar WordPress usando WP-CLI
27     wp core install \
28         --url="$WP_URL" \
29         --title="$WP_TITLE" \
30         --admin_user="$WP_ADMIN_USER" \
31         --admin_password="$WP_ADMIN_PASSWORD" \
32         --admin_email="$WP_ADMIN_EMAIL" \
33         --path="/var/www/wordpress"
34 fi
35
36 # Arrancar PHP-FPM
37 php-fpm
38

```

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Aceves Siordia	05/10/2025
	Nombre: Kevin Damian	

Por otro lado, para la imagen de Nginx se usará la imagen oficial como base, realizando únicamente la modificación de precargar el archivo de configuración del sitio *nginx.conf*. Esta decisión como la anterior sigue un enfoque de “two-way door”, ya que ofrece las dos posibilidades:

- Usar la configuración predeterminada incluida en la imagen personalizada, asegurando que funcione sin tener que configurar algo mas
- Sobrescribir el archivo de configuración permitiendo ajustes o personalizaciones sin necesidad de reconstruir la imagen

```

100, 3 days ago | 1 author (100)
1  # Imagen base de Nginx
2  FROM nginx:latest
3
4  # Copiar la configuracion
5  COPY nginx.conf /etc/nginx/conf.d/default.conf
6
7  # Exponemos el puerto 80
8  EXPOSE 80
9
10 # No es necesario un CMD ya que la imagen de nginx viene con un CMD que es heredado
    debido a que esta imagen no tiene CMD o ENTRYPOINT definido
    You, 3 days ago • Con

```

Despliegue

El proceso de despliegue de los contenedores es el siguiente, esto para garantizar que cada componente quedara correctamente configurado antes de pasar al siguiente:

1. **Creación de la red:** Se definió una red bridge en Docker que actuara como canal de comunicación entre los contenedores de esta solución.

```

PS D:\Documents\ProjectsProg\MastersDevOps\S2\Contenedores\Actividad1> docker network create wp-network
2324691485ecb6d6a46474094987139f179a94c7470c86355bb4498a85bd2a19

```

2. **Creación del volumen de la base de datos:** Se genero un volumen persistente que permite conservar la información incluso si el contenedor es detenido, eliminado o reemplazado. Potencialmente permitiría el respaldo de este.

```

PS D:\Documents\ProjectsProg\MastersDevOps\S2\Contenedores\Actividad1> docker volume create wp-db-data
wp-db-data

```

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Aceves Siordia	05/10/2025
	Nombre: Kevin Damian	

- Puesta en marcha de la base de datos:** Se despliega el contenedor de MySQL configurando las credenciales y parámetros de inicialización. Además de vincular a la red y volumen que creamos anteriormente.

```
PS D:\Documents\ProjectsProg\MastersDevOps\S2\Contenedores\Actividad1> docker run -d --name wp-db --network wp-network -v wp-data-data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=rootpassword -e MYSQL_DATABASE=wordpress -e MYSQL_USER=wpuser -e MYSQL_PASSWORD=wppassword mysql:8.1
99467be007021b038c0be999e5e633f471447cc8d1b74d0aeb0e2141801c0502
```

- Puesta en marcha de WordPress:** Primero se construye la imagen con el Dockerfile y entryptpoint, y se tagueo bajo un nombre. Una vez construida se pone en marcha pasando por variables de ambiente las credenciales de la base de datos y del título del sitio (aunque esto retendrá algunos de los valores por default del entryptpoint) así como vincularla a la red.

```
PS D:\Documents\ProjectsProg\MastersDevOps\S2\Contenedores\Actividad1\WordPress> docker build -t my-wordpress .
[+] Building 39.3s (14/14) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.25kB
=> [internal] load metadata for docker.io/library/php:8.2-fpm
=> [auth] library/php:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> CACHED [1/8] FROM docker.io/library/php:8.2-fpm@sha256:d71ca261f13e4c11dbffcd5a2b31bc51a10ac0e4c16f9f6c6214012be726c9de
=> => resolve docker.io/library/php:8.2-fpm@sha256:d71ca261f13e4c11dbffcd5a2b31bc51a10ac0e4c16f9f6c6214012be726c9de
=> [internal] load build context
=> => transferring context: 1.28kB
=> [2/8] RUN apt-get update && apt-get install -y default-mysql-client libzip-dev libonig-dev libxml2-dev curl tar
=> [3/8] RUN mkdir -p /var/www && chown www-data:www-data /var/www
=> [4/8] RUN curl -sSL https://wordpress.org/latest.tar.gz | tar -xz -C /var/www && chown -R www-data:www-data /var/www/wordpress
=> [5/8] RUN curl -O https://raw.githubusercontent.com/wp-cli/builds/gh-pages/phar/wp-cli.phar && chmod +x wp-cli.phar && mv wp-cli.ph
=> [6/8] COPY docker-entrypoint.sh /usr/local/bin/docker-entrypoint.sh
=> [7/8] RUN chmod +x /usr/local/bin/docker-entrypoint.sh
=> [8/8] WORKDIR /var/www/wordpress
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:311073691e845452ac6af4614f9896c5227830df644f6bef7544c84f24be922
=> => exporting config sha256:5835e4d6fe8d066790286565214d5c5a92a8c6803d3ff7ed70aad27cb09582bb
=> => exporting attestation manifest sha256:7f4bd43dd010ac50b3a8df5a0ebe3029fc95010f6f78413481199b523a24f31c
=> => exporting manifest list sha256:6af35f299cf3683ac2485f532a7f40e8c06b3afd543bf3c3797341bb8af8af0b
=> => naming to docker.io/library/my-wordpress:latest
=> => unpacking to docker.io/library/my-wordpress:latest
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/xmhtfn2nhrt3ny2wptg8c313n
PS D:\Documents\ProjectsProg\MastersDevOps\S2\Contenedores\Actividad1\WordPress> docker run -d --name wp-php --network wp-network -e DB_HOST=wp-db -e DB_USER=wpuser -e DB_PASSWORD=wppassword -e DB_NAME=wordpress -e WP_TITLE=KevinAceves my-wordpress
c00dc3d5c31b6fdc569ab1a33433f267c76e6d44afa5b889fb970a4eb4471c62
```

- Puesta en marcha de Nginx:** Como en WordPress, primero se crea la imagen tageada en base al Dockerfile y archivo de configuración. Después, se pone en marcha, vinculándolo a la misma red y haciendo port forwarding del puerto 80 del contenedor al 8080 de la computadora, permitiendo que se pueda acceder desde el navegador yendo al localhost:8080.

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Aceves Siordia	05/10/2025
	Nombre: Kevin Damian	

```

PS D:\Documents\ProjectsProg\MastersDevOps\S2\Contenedores\Actividad1\Nginx> docker build -t my-nginx .
[+] Building 1.1s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 343B
=> [internal] load metadata for docker.io/library/nginx:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 32B
=> [1/2] FROM docker.io/library/nginx:latest@sha256:75cad0ed4b9ea94413143f700eea1a71635806207ffb901b577c436a06ff4acd
=> => resolve docker.io/library/nginx:latest@sha256:75cad0ed4b9ea94413143f700eea1a71635806207ffb901b577c436a06ff4acd
=> [auth] library/nginx:pull token for registry-1.docker.io
=> CACHED [2/2] COPY nginx.conf /etc/nginx/conf.d/default.conf
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:600a26774c106ba73a46c336b40bd8fa6da933f8751e59a42edad3695dc13c5f
=> => exporting config sha256:69897be733b77d3f6f809ccac461e9ed3afc39c4c4077930ea0d2f36fb566fffb
=> => exporting attestation manifest sha256:9a4fa641c0a67f7319b00c285988acfabace50aca84237428d66fa5c7fb331a6
=> => exporting manifest list sha256:b85e0f04a3b56b3f6d40dd7e60208697cbc8985add37c968a34e6c41d08854a3
=> => naming to docker.io/library/my-nginx:latest
=> => unpacking to docker.io/library/my-nginx:latest
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/okw7zefd8iiy09g8hyagbvpbw

PS D:\Documents\ProjectsProg\MastersDevOps\S2\Contenedores\Actividad1\Nginx> docker run -d -p 8080:80 --name wp-nginx --network wp-network my-nginx
4cb88e28042116e638a14c207423063ba72c25b447c01bef174ba30413614569

```

6. **Validación de la red:** Se verifica que los tres contenedores estén conectados a la red y que puedan resolverse entre si por el nombre del contenedor

```

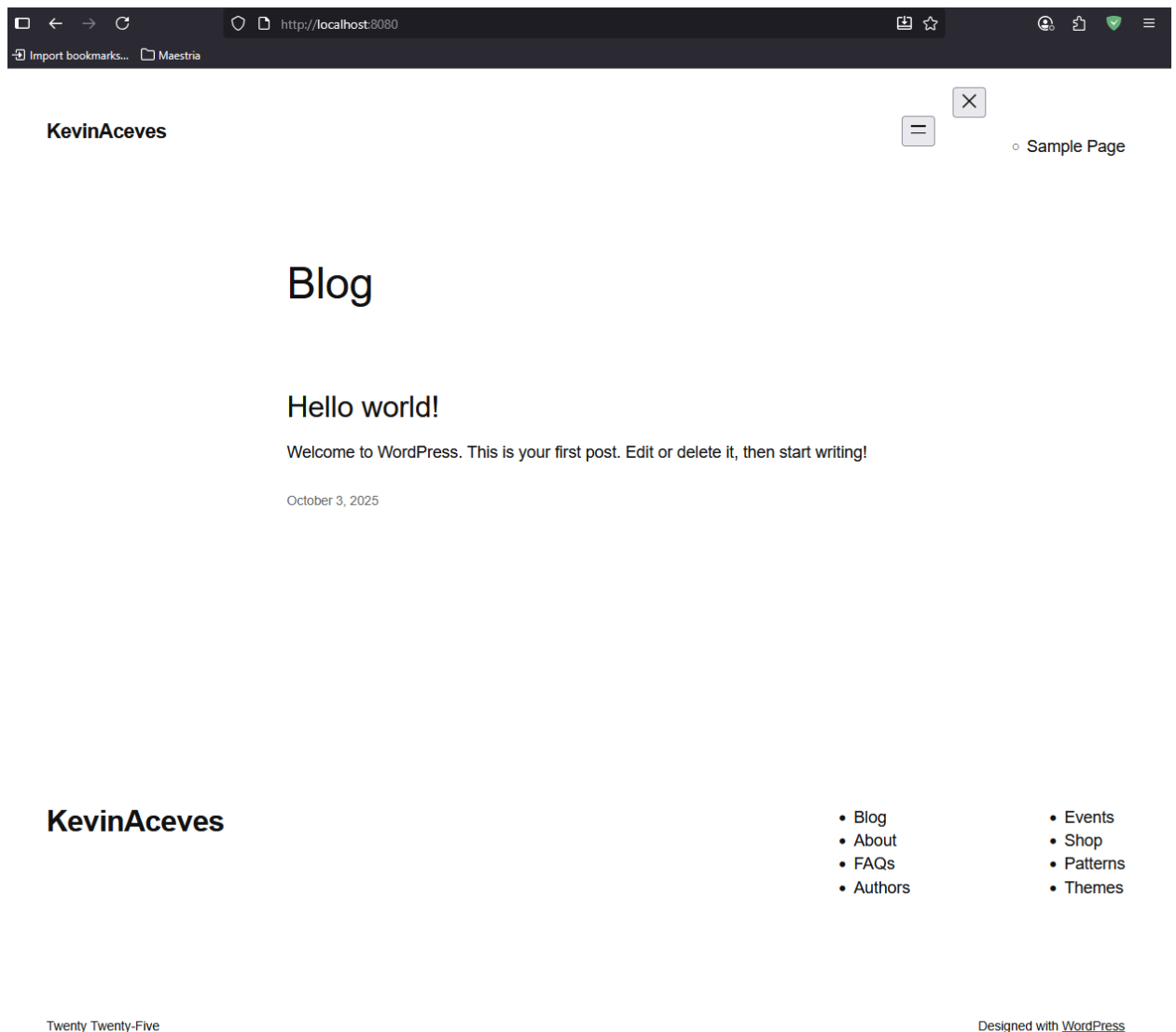
PS D:\Documents\ProjectsProg\MastersDevOps\S2\Contenedores\Actividad1\Nginx> docker network inspect wp-network

{
  "Name": "wp-network",
  "Subnet": "172.18.0.0/16",
  "Gateway": "172.18.0.1",
  "IPAM": {
    "Driver": "bridge",
    "Options": {
      "BridgeName": "veth",
      "Auxiliary": {}
    }
  },
  "Internal": false,
  "Attachable": false,
  "Ingress": false,
  "ConfigFrom": {
    "Network": ""
  },
  "ConfigOnly": false,
  "Containers": {
    "4cb88e28042116e638a14c207423063ba72c25b447c01bef174ba30413614569": {
      "Name": "wp-nginx",
      "EndpointID": "a399d8dcf0d54d7cc74ba44ff9a1cde84025965087aad51686ccadef278fed5c",
      "MacAddress": "86:9f:63:43:75:de",
      "IPv4Address": "172.18.0.4/16",
      "IPv6Address": ""
    },
    "99467be007021b038c0be999e5e633f471447cc8d1b74d0aeb0e2141801c0502": {
      "Name": "wp-db",
      "EndpointID": "2d1900b95764e083a78c7dac3bb7ba3040dc31731d257ec3af8a09dbff3cf432",
      "MacAddress": "b6:65:28:74:6f:08",
      "IPv4Address": "172.18.0.2/16",
      "IPv6Address": ""
    },
    "c00dc3d5c31b6fdc569ab1a33433f267c76e6d44afa5b889fb970a4eb4471c62": {
      "Name": "wp-php",
      "EndpointID": "03f9b44ddd4299effcd0bd47ae7a548c3657ad666e6d8d3460f8185abe52d53b",
      "MacAddress": "42:f1:d9:a8:8b:3a",
      "IPv4Address": "172.18.0.3/16",
      "IPv6Address": ""
    }
  },
  "Options": {},
  "Labels": {}
}

```

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Aceves Siordia	05/10/2025
	Nombre: Kevin Damian	

7. **Acceso desde el navegador:** Probar que la aplicación se ha lanzado y configurado correctamente ingresando en el navegador a la dirección <http://localhost:8080>. A continuación, una captura de pantalla del resultado, mostrando la página inicial de WordPress ya configurada.



Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Aceves Siordia	05/10/2025
	Nombre: Kevin Damian	

Subir a DockerHub

Una vez generadas las imágenes personalizadas que hemos creado, el siguiente paso sería publicarlas en DockerHub para garantizar su disponibilidad y reutilización por cualquier otra persona.

- 1) Iniciar sesión en DockerHub: En mi caso al trabajar desde un Windows con Docker Desktop, la sesión se encontraba iniciada, pero se puede usar el comando *docker login* en caso de que no se tenga.
- 2) Etiquetado y publicación: Se utiliza el comando *docker tag* para asociar las imágenes locales con el repositorio de DockerHub. Después se usa el comando *docker push* para subir las imágenes

```

PS D:\Documents\ProjectsProg\MastersDevOps\S2\Contenedores\Actividad1\Nginx> docker tag my-wordpress:latest fadeguy/my-wordpress
PS D:\Documents\ProjectsProg\MastersDevOps\S2\Contenedores\Actividad1\Nginx> docker push fadeguy/my-wordpress
Using default tag: latest
The push refers to repository [docker.io/fadeguy/my-wordpress]
5156a37740f5: Pushed
8c7716127147: Pushed
363c276d0dc2: Pushed
2489d5e860a7: Pushed
e1cf44d6017a: Pushed
5743de172d68: Pushed
4f4fb700ef54: Pushed
7cdee73376b7: Pushed
24403a1f6855: Pushed
a06069021870: Pushed
ef3468b7c908: Pushed
ca5623a61e9c: Pushed
836b4fbe4b6f: Pushed
836c8e3f5daf: Pushed
c88cb275c739: Pushed
f602367faaea: Pushed
3ba33e986f5a: Pushed
888f825138e7: Pushed
latest: digest: sha256:6af35f299cf3683ac2485f532a7f40e8c06b3afd543bf3c3797341bb8af8af0b size: 856

```

```

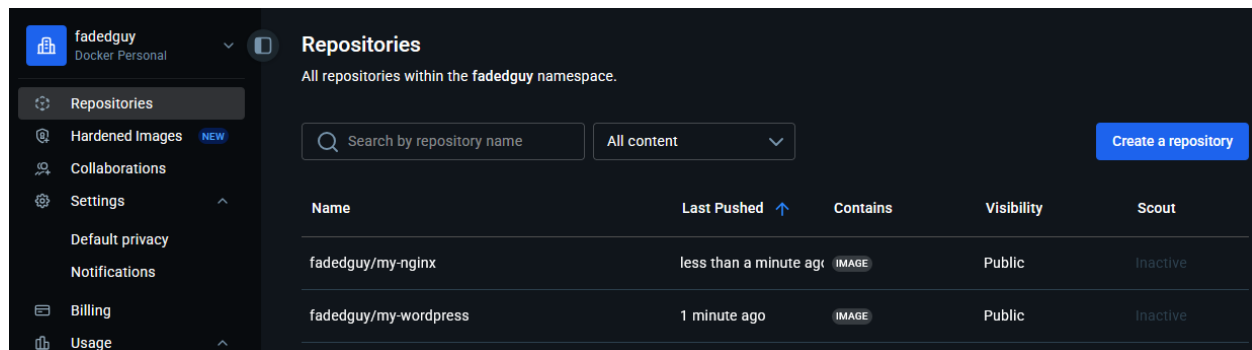
PS D:\Documents\ProjectsProg\MastersDevOps\S2\Contenedores\Actividad1\Nginx> docker tag my-nginx:latest fadeguy/my-nginx
PS D:\Documents\ProjectsProg\MastersDevOps\S2\Contenedores\Actividad1\Nginx> docker push fadeguy/my-nginx
Using default tag: latest
The push refers to repository [docker.io/fadeguy/my-nginx]
3cc5fdd1317a: Mounted from library/nginx
5f825f15e2e0: Mounted from library/nginx
4f4e50e20765: Mounted from library/nginx
16d05858bb8d: Mounted from library/nginx
e7567cf67efd: Pushed
5c32499ab806: Mounted from library/nginx
375a694db734: Mounted from library/nginx
08cfef42fd24: Mounted from library/nginx
a8ec84e2e803: Pushed
latest: digest: sha256:b85e0f04a3b56b3f6d40dd7e60208697cbc8985add37c968a34e6c41d08854a3 size: 856

```

- 3) Verificación: Se accedió al portal de DockerHub para confirmar que las imágenes habían sido cargadas correctamente y quedaran disponibles para su uso desde cualquier otra máquina.

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Aceves Siordia	05/10/2025
	Nombre: Kevin Damian	

- Nginx: <https://hub.docker.com/repository/docker/fadedguy/my-nginx>
- WordPress: <https://hub.docker.com/repository/docker/fadedguy/my-wordpress>



Conclusiones

En esta actividad se aplicaron los fundamentos de la contenerización con Docker para desplegar todos los componentes necesarios de WordPress de manera desacoplada.

En comparación con montarlo en una máquina virtual (como fue el caso en el cuatrimestre pasado), las ventajas que Docker ofrece en términos de simplicidad, rapidez, flexibilidad, portabilidad y eficiencia se hicieron evidentes durante la creación, ejecución y administración del entorno.